

Chapter 3. MLflow for Traditional ML

MLflow is an open-source tool that was originally created by Databricks for experiment tracking and model registry, and it's one of the most popular tools in the MLOps ecosystem. It is available both as a standalone tool and as an integrated component of the Databricks platform.

Early in my career, before MLflow existed, we built our own internal tooling to store metrics, parameters, and models. It worked, but it was painful to maintain and hard to standardize across teams. When MLflow came along, it solved exactly that problem, and it has only grown since.

MLflow provides a wide range of features, for both standard machine learning models and LLM-based applications. Here's a list of the most important ones (which we'll also cover in [Chapter 8](#)):

MLflow Tracking

This is a centralized repository that logs ML model parameters, metrics, artifacts, and a reference to data used for model training. Experiment tracking gives insights into changes in models over time, and makes it possible for you to search for runs and experiments, so it's crucial for traceability and reproducibility.

MLflow Model Registry

This is a centralized model store that allows users to manage a model's lifecycle and add tags, aliases, and new model versions. It has a different backend than experiment tracking. To register a model, you must first log it using MLflow Tracking. On Databricks, you should use the Unity Catalog model registry to register models.

MLflow evaluation

This is a set of tools that facilitate model evaluation and model comparison. In traditional ML applications, you can use it to compare a new model against the baseline model or a threshold. MLflow supports a range of metrics for LLM evaluation (such as answer correctness, answer relevance, and faithfulness), as well as custom metrics that a user can define.

This is a feature that is integrated into MLflow Tracking and that enhances the observability of GenAI applications by recording inputs, outputs, and metadata associated with each step within the application. This allows users to pinpoint the source of unexpected behavior when it occurs.

This book focuses on the Databricks-managed version of MLflow, which differs in several ways from the open-source project. As a result, instead of covering features like the MLflow AI Gateway and MLflow Model Serving in their open-source form, we'll highlight the corresponding alternatives that are available within the Databricks platform.

We'll begin by exploring *experiment tracking*, which is one of the most mature and widely used components of MLflow. It serves as the foundation for managing ML model runs.

Experiment Tracking

Experiment tracking helps teams keep track of model performance, configuration, and results across iterations. As ML projects grow in complexity, it gets easy to lose track of which model version was trained with which parameters or dataset.

MLflow addresses this challenge with its experiment tracking component. In MLflow, *experiments* serve as the primary unit of organization for model training runs. Every *run*, which captures metadata such as parameters, metrics, artifacts, and source data, is associated with an experiment. That makes it easy to compare and reproduce results.

Tracking URI

By default, MLflow will track experiment runs using the local file system, and all the metadata will be stored in the `./mlruns` directory. Since we use the Databricks-managed version of MLflow, you need to set tracking URI to `"databricks"` when running code locally so that all the experiment runs will be tracked within a Databricks workspace. Even though experiment tracking does not use registry URI, starting with MLflow version 3, you still need to set the MLflow registry URI to `"databricks-uc"`.

You do this by setting the **MLFLOW_TRACKING_URI** and **MLFLOW_REGISTRY_URI** environment variables or running the `mlflow.set_tracking_uri()` and `mlflow.set_registry_uri()`

commands. Please note that you need to provide the profile you used to authenticate to Databricks in the tracking and registry URI. These profiles are defined in the `.databrickscfg` file (an example of the profile name could be `mlops-book`). If you use the `DEFAULT` profile, you do not need to pass the profile name to the tracking URI:

```
import mlflow

profile="mlops-book"
mlflow.set_tracking_uri(f"databricks://{profile}")
mlflow.set_registry_uri(f"databricks-uc://{profile}")
```

If we run the code on Databricks, however, we don't need to set the tracking URI. We also want multiple developers with a different profile name to collaborate on the same code base. The solution would be to store the profile name in an `.env` file (which should be ignored by Git via a `.gitignore` file) and set the `PROFILE` environment variable by using the [python-dotenv package](#).

We can then define whether the code runs on Databricks by checking whether or not the `DATABRICKS_RUNTIME_VERSION` environment variable is available. We can define that function under the common module of the custom package (`src/hotel_booking/utils/common.py`) and use this code in all the scripts that use MLflow:

```
import os

import mlflow
from dotenv import load_dotenv

def set_mlflow_tracking_uri() -> None:
    if "DATABRICKS_RUNTIME_VERSION" not in os.environ:
        load_dotenv()
        profile = os.environ["PROFILE"]
        mlflow.set_tracking_uri(f"databricks://{profile}")
        mlflow.set_registry_uri(f"databricks-uc://{profile}")
```

We can then check the current tracking and registry URI by using the `mlflow.get_tracking_uri()` and `mlflow.get_registry_uri()` commands. After we have set the tracking URI, we can define the MLflow experiment.

MLflow Experiment

You can create an MLflow experiment by using the

`mlflow.create_experiment()` or `mlflow.set_experiment()` function. The `mlflow.set_experiment()` function will activate the existing experiment by name or will create a new one if such an experiment does not exist yet:

```
mlflow.set_experiment(experiment_name="/Shared/demo")
```

You can add tags to your experiment by passing it as a parameter to the `mlflow.create_experiment()` command, or if the experiment already exists, you can use the `mlflow.set_experiment_tags()` command after the experiment gets activated:

```
mlflow.set_experiment_tags(  
    {"repository_name": "mvechtomova/oreilly-mlops-with-databricks"})
```

You can then retrieve an experiment by its experiment id, by using the `mlflow.get_experiment()` command as follows:

```
mlflow.get_experiment(experiment.experiment_id)
```

You can also search for the MLflow experiment by name or by tag. The `search_experiments()` function outputs a list of instances of an experiment class that contains information about the experiment, such as its `experiment_id`, creation time, and artifact location, plus tags created by Databricks and by the users:

```
experiments = mlflow.search_experiments(  
    filter_string="tags.repository_name='mvechtomova/oreilly-mlops-with-databricks'"  
)
```

You can also easily view how the class is constructed. First, you save all class attributes in a JSON file:

```
import json  
import os  
  
if not os.path.exists("../demo_artifacts"):  
    os.mkdir("../demo_artifacts")
```

```
with open("../demo_artifacts/mlflow_experiment.json", "w") as json_file:
    json.dump(experiment.__dict__, json_file, indent=4)
```

Then, you visualize it with [JSON Lens](#) or a similar Visual Studio Code extension (see [Figure 3-1](#)).

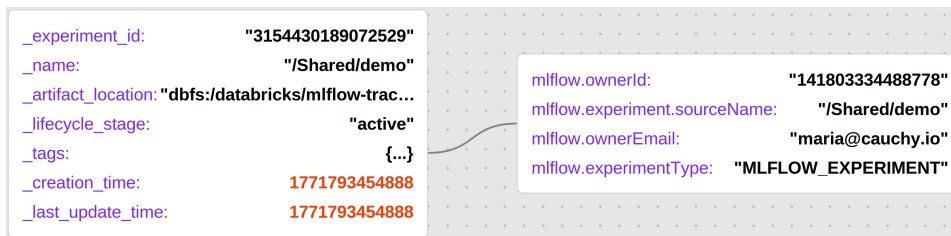


Figure 3-1. Visualization of the `mlflow.entities.experiment.Experiment` class attributes

MLflow Run

We can create an MLflow run by running the `mlflow.start_run()` command. Let's start a run and log some metrics and parameters:

```
with mlflow.start_run(
    run_name="demo-run",
    tags={"git_sha": "1234567890abcd",
         "branch": "main"},
    description="demo run",
) as run:
    mlflow.log_params({"type": "demo"})
    mlflow.log_metrics({"metric1": 1.0, "metric2": 2.0})
```

Note that we use the command within the `with` block. If you didn't use it, you would have to run the `mlflow.end_run()` command to end the run. In this run, we logged a parameter and some metrics.

We can retrieve the run by using the `mlflow.get_run()` command:

```
run_info = mlflow.get_run(run_id=run_id)
```

You can easily visualize how the MLflow run class is constructed. First, save the run information as a JSON file:

```
run_info_dict = run_info.to_dictionary()
with open("../demo_artifacts/run_info.json", "w") as json_file:
    json.dump(run_info_dict, json_file, indent=4)
```

Then, visualize it with JSON Lens (see [Figure 3-2](#)).

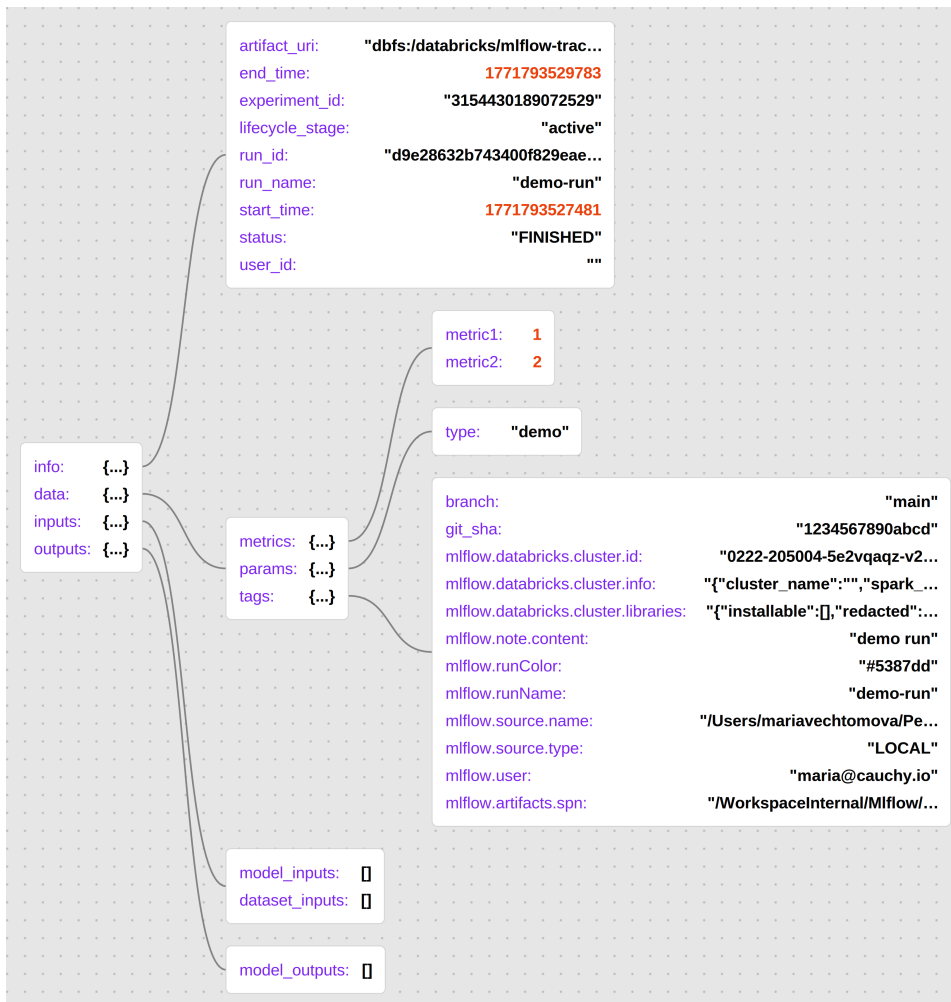


Figure 3-2. Visualization of the `mlflow.entities.run.Run` class attributes

We can access the logged metrics and parameters via the dictionary that contains run information that we defined earlier:

```
run_info_dict["data"]["metrics"]
run_info_dict["data"]["params"]
```

And we can also search for a run by using the `mlflow.search_runs()` function:

```
from time import time

time_hour_ago = int((time() - 3600) * 1000)

runs = mlflow.search_runs(
    search_all_experiments=True, # or experiment_ids=[], or experiment_names=[]
    order_by=["start_time DESC"],
    filter_string="status='FINISHED' AND "
    f"start_time>{time_hour_ago} AND "
    "run_name LIKE '%demo-run%' AND "
```

```
"metrics.metric3>0"  
)
```

The function supports complex string filtering, which we can use to filter by tag, metrics, run name, start time, and nearly any other data related to the run.

We can also start the existing run again and log extra parameters, metrics, and artifacts. Note that we can't overwrite the old data. For example, if `metric1` already exists and has a value of 1.0, we cannot assign a value of 2.0 to it.

Let's start the run again and log some artifacts and metrics:

```
mlflow.start_run(run_id=run_id)  
  
for i in range(0,3):  
    mlflow.log_metric(key="metric1", value=3.0+i/2, step=i)  
    mlflow.log_artifact("../demo_artifacts/mlflow_meme.jpeg")  
    mlflow.log_text("hello, MLflow!", "hello.txt")  
    mlflow.log_dict({"k": "v"}, "dict_example.json")  
    mlflow.log_artifacts("../demo_artifacts", artifact_path="demo_artifacts")
```

We can log metrics dynamically, like during each training epoch. We can log any file using the `mlflow.log_artifact()` command. To log a folder, we can use the `mlflow.log_artifacts()` function instead. We can also log text and a dictionary with a dedicated command.

MLflow experiment tracking supports logging images and figures. You can even log images dynamically, for example, after each training epoch, or in a loop like in the following example. Since you're not using a `with` block, remember to end the run once you've finished logging:

```
import matplotlib.pyplot as plt  
import numpy as np  
  
fig, ax = plt.subplots()  
ax.plot([0, 1], [2, 3])  
mlflow.log_figure(fig, "figure.png")  
  
for i in range(0,3):  
    image = np.random.randint(0, 256, size=(100, 100, 3), dtype=np.uint8)  
    mlflow.log_image(image, key="demo_image", step=i)  
  
mlflow.end_run()
```

At this point, we've covered the most important logging functionalities that are relevant to ML projects, except logging the models. We'll get to that in the next section of this chapter. All the functionalities we've discussed so far weren't marked as experimental at the time of writing. Earlier, we showed you how to retrieve parameters and metrics; now, let's look at how to retrieve logged artifacts.

First of all, we need to retrieve the artifact URI, which we must do if we want to use the `load_dict()`, `load_image()`, and `load_text()` functions from the `mlflow.artifacts` module:

```
artifact_uri = runs.artifact_uri[0]
dict_example = mlflow.artifacts.load_dict(
    f"{artifact_uri}/dict_example.json")
figure = mlflow.artifacts.load_image(f"{artifact_uri}/figure.png")
text = mlflow.artifacts.load_text(f"{artifact_uri}/hello.txt")
```

Then, download artifacts using the `mlflow.artifacts.download_artifacts()` function:

```
if not os.path.exists("../downloaded_artifacts"):
    os.mkdir("../downloaded_artifacts")
mlflow.artifacts.download_artifacts(
    artifact_uri=f"{artifact_uri}/demo_artifacts",
    dst_path="../downloaded_artifacts")
```

[Figure 3-3](#) demonstrates all the logging and loading functionality for MLflow.

Log		Load/download	
start_run Start a new run	<code>mlflow.start_run()</code>	get_run Load a run by id	<code>mlflow.get_run()</code>
log_param Log a single parameter	<code>mlflow.log_param("learning_rate", 0.01)</code>	Access a single parameter	<code>run["data"]["params"]["learning_rate"]</code>
log_params Log multiple parameters	<code>mlflow.log_params({"lr": 0.01, "n_estimators": 100})</code>	Access all parameters	<code>run["data"]["params"]</code>
log_metric Log a single metric	<code>mlflow.log_metric("mse", 2500.0)</code>	Access a single metric	<code>run["data"]["metrics"]["mse"]</code>
log_metrics Log multiple metrics	<code>mlflow.log_metrics({"mse": 2500.0, "rmse": 50.0})</code>	Access all metrics	<code>run["data"]["metrics"]</code>
log_artifact Log a single artifact (file)	<code>mlflow.log_artifact("mlflow_demo.py")</code>	download_artifacts Download a single artifact	<code>mlflow.artifacts.download_artifacts(uri)</code>
log_artifacts Log a directory of artifacts	<code>mlflow.log_artifacts("demo/")</code>	download_artifacts Download a directory	<code>mlflow.artifacts.download_artifacts(uri, dst_path="demo")</code>
log_dict Save a dictionary as JSON/YAML	<code>mlflow.log_dict({"k": "v"}, "dict_example.json")</code>	load_dict Load a dict	<code>mlflow.artifacts.load_dict(f"{url}/dict_example.json")</code>
log_text Save a string as a text file	<code>mlflow.log_text("hello", "hello.txt")</code>	load_text Load text	<code>mlflow.artifacts.load_text(f"{url}/hello.txt")</code>
log_figure Save a matplotlib/plotly figure	<code>fig, ax = plt.subplots() ax.plot([1, 2, 3], [4, 5, 6]) mlflow.log_figure(fig, "figure.png")</code>	load_image Load an image (figure)	<code>mlflow.artifacts.load_image(f"{url}/figure.png")</code>
log_image Save a numpy/PIL image	<code>from PIL import Image img = Image.open("x.png") mlflow.log_image(img, "image.png")</code>	load_image Load an image	<code>mlflow.artifacts.load_image(f"{url}/figure.png")</code>

Figure 3-3. MLflow logging and loading functionality

All the logged information can be also found in the UI. [Figure 3-4](#) demonstrates the top-level overview of the run. There, we can find logged metrics and parameters, together with the information about the run and logged artifacts.

model performance. Therefore, we'll use *time-based splits*, which means that we'll train on 12 months of data, use the subsequent month for validation (we'll use the validation set in the hyperparameter tuning), and use one month after that for testing.

In general, whenever possible, you should use time-based splits. For example, you should train your model on two weeks of data and evaluate it on the following week. This is the better approach because some features, like product recommendations that change with the seasons, may be time-correlated in ways that aren't immediately obvious.

This works for datasets in which the timestamp of data generation is known. But sometimes, it's not, and we've often seen that the split is redefined every time the model is retrained. This leads to *data leakage*: IDs from the train set end up in the test set and vice versa. A random state won't help. A better approach in this case is to fix the train-test split from the start and keep it separate. Then, when new data arrives, it will be divided between the training and testing sets without any reshuffling.

Now, let's define a `DataLoader` class under the `hotel_booking.data` module:

```
from datetime import datetime, timedelta

import pandas as pd
from dateutil.relativedelta import relativedelta
from pyspark.sql import SparkSession

from hotel_booking.config import ProjectConfig
from hotel_booking.utils.common import get_delta_table_version


class DataLoader:
    """Loads, splits, and accesses datasets in both Spark and pandas formats."""

    def __init__(self, config: ProjectConfig, spark: SparkSession) -> None:
        """
        :param config: Project configuration containing table and feature info.
        :param spark: Active Spark session.
        """
        self.cfg = config
        self.spark = spark
```

The `split` method outputs `X_train`, `y_train`, `X_test`, and `y_test`, which belong to the right time period from the `hotel_booking` table

and are used further to train the model. The split method accepts an optional version argument, which refers to the version of the delta table:

```
def split(self, test_months: int = 1, train_months: int = 12,
          max_date: datetime = None, version: str = None,
) -> tuple[pd.DataFrame, pd.Series, pd.DataFrame, pd.Series]:
    """
    #Splits the dataset and loads it into pandas DataFrames for training and tes

    :param test_months: Number of months for the test set.
    :param train_months: Number of months for the training set.
    :param max_date: The maximum date for splitting. If None, uses the latest da
    :param version: Optional Delta table version to query.
    :return: A tuple (X_train, y_train, X_test, y_test) with pandas DataFrames.
    """
    self.train_query, self.test_query = self.generate_queries(
        test_months, train_months, max_date, version)

    self.train_set_spark = self.spark.sql(self.train_query)
    self.test_set_spark = self.spark.sql(self.test_query)

    train_set = self.train_set_spark.toPandas()
    test_set = self.test_set_spark.toPandas()

    X_train = train_set[self.cfg.num_features + self.cfg.cat_features]
    y_train = train_set[self.cfg.target]

    X_test = test_set[self.cfg.num_features + self.cfg.cat_features]
    y_test = test_set[self.cfg.target]

    return X_train, y_train, X_test, y_test
```

The split() method relies on generate_queries() to construct queries for reading the training and test datasets. These queries are based on the start and end dates of the training and testing periods, which are calculated by _get_start_end_dates() using the provided period length and max_date. If the max_date is not specified, the latest available date will be retrieved from the hotel_booking table. Similarly, if no Delta table version is provided, the most recent version will be obtained from the table history:

```
def generate_queries(
    self, test_months: int = 1, train_months: int = 12,
    max_date: datetime = None, version: int = None,
) -> tuple[str, str]:
    """
    #Generates SQL queries for the training and test splits.
```

```

:param test_months: Number of months for the test set.
:param train_months: Number of months for the training set.
:param max_date: The maximum date for splitting. If None, uses the max date.
:param version: Optional Delta table version to query.
:return: A tuple containing the training and test SQL queries.
"""

table_ref = f"{self.cfg.catalog}.{self.cfg.schema}.hotel_booking"

if max_date is None:
    max_date = self.spark.sql(
        f"SELECT MAX(date_of_reservation) AS max_date FROM {table_ref}"
    ).collect()[0]["max_date"]

if version is None:
    version = get_delta_table_version(self.spark, table_ref)

train_start, train_end, test_start, test_end = self._get_start_end_dates(
    max_date, test_months, train_months)

train_query = f"""
    SELECT * FROM {table_ref} VERSION AS OF {version}
    WHERE arrival_date BETWEEN DATE('{train_start}') AND DATE('{train_end}')
"""

test_query = f"""
    SELECT * FROM {table_ref} VERSION AS OF {version}
    WHERE arrival_date BETWEEN DATE('{test_start}') AND DATE('{test_end}')
"""

return train_query, test_query

```

We won't go into the implementation details of

`_get_start_end_dates()` and `get_delta_table_version()` from the common module here; refer to the code for more details. Most importantly, the generated queries will allow us to get the exact version of data used for training, even if data is modified later, thanks to the time travel functionality of delta tables.

NOTE

Remember to set a proper retention period on the delta table (the default is seven days). Otherwise, you may not be able to access the exact version of the table if the `VACUUM` command has been executed. Most accounts have predictive optimization enabled by default, which means that Databricks automatically triggers the `VACUUM` operation as part of its optimization process.

Notice that the `split` method uses PySpark to read the Delta table and then converts the PySpark DataFrame to Pandas, which is required as input for the LightGBM model that we will train later. We chose PySpark here for simplicity and because it is currently the easiest way to access data stored in Delta tables.

However, for large datasets, this conversion is not very efficient. A faster alternative is to use credential vending together with `pyarrow` and the `deltalake` library, but that approach requires a more complex setup. Additionally, we will need a PySpark DataFrame later to log the input data within the MLflow run.

Now, we need to load a training and validation set for hyperparameter tuning (where the validation set ends on 2018-11-30). You can find the code in the `chapter3_2.model_finetuning.py` notebook:

```
import os
from datetime import datetime

import pandas as pd
from pyspark.sql import SparkSession

from hotel_booking.config import ProjectConfig
from hotel_booking.data.data_loader import DataLoader
from hotel_booking.utils.common import set_mlflow_tracking_uri

set_mlflow_tracking_uri()

cfg = ProjectConfig.from_yaml(config_path="../project_config.yml")

spark = SparkSession.builder.getOrCreate()
data_loader = DataLoader(spark=spark, config=cfg)

X_train, y_train, X_valid, y_valid = data_loader.split(
    test_months=1,
    train_months=12,
    max_date=datetime.strptime("2018-11-30", "%Y-%m-%d")
)
```

Now that we have loaded the data, let's see how we can use nested MLflow runs for hyperparameter tuning.

Hyperparameter Tuning and Nested

Runs

Every ML project starts with setting a business problem, analyzing the data, choosing a model, and tuning the hyperparameters. However, hyperparameter tuning does not just happen at the beginning of the project. It's important to keep reevaluating hyperparameters periodically when a model is in production. However, hyperparameter tuning is an expensive exercise, and it should not occur every time the model gets retrained.

There are several popular libraries for hyperparameter tuning, including [Hyperopt](#), [Optuna](#), and [Ray Tune](#). Historically, Hyperopt was the go-to solution, and it's still widely referenced in older examples and blog posts. However, the ecosystem has evolved, and Hyperopt is no longer maintained.

Today, it's important to distinguish between the optimization algorithm and the orchestration layer. Optuna provides state-of-the-art hyperparameter search algorithms such as [Tree-structured Parzen Estimator](#) (TPE), while Ray Tune focuses on scalable and distributed execution of tuning workloads.

In this chapter, we use Ray Tune together with Optuna: Optuna defines the search strategy, and Ray Tune handles parallelization and distributed execution. This approach reflects current best practices for running large-scale tuning workloads on Databricks with MLflow.

MLflow nested runs are especially convenient for hyperparameter tuning: all the child runs will be grouped under one parent run, which will help with comparison and evaluation. Here is the most basic example of MLflow nested runs:

```
with mlflow.start_run(run_name="top_level_run") as run:
    for i in range(1,5):
        with mlflow.start_run(run_name=f"subrun_{str(i)}", nested=True) as subrun:
            mlflow.log_metrics({"m1": 5.1+i,
                               "m2": 2*i,
                               "m3": 3+1.5*i})
```

Next, let's demonstrate how we can use MLflow nested runs with Ray Tune for our project. Earlier, we defined the training and validation sets. Let's now go through the model logic. You can find all the code in the *chapter3_2.model_finetuning.py* notebook.

The `LightGBMModel` class is defined in the `models` module of the `hotel_booking` package:

```
from datetime import datetime

import mlflow
import pandas as pd
import pyspark
from lightgbm import LGBMRegressor
from loguru import logger
from mlflow import MlflowClient
from mlflow.models import infer_signature
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from hotel_booking.config import ProjectConfig, Tags


class LightGBMModel:
    """A basic model class for house price prediction using LightGBM.

    This class handles data loading, feature preparation,
    model training, and MLflow logging.
    """

    def __init__(self, config: ProjectConfig) -> None:
        """Initialize the LightGBM model."""
        self.cfg = config
        self.cat_features = self.cfg.cat_features
        self.parameters = self.cfg.parameters.model_dump()
```

In the `train` method, we define a `sklearn` pipeline that consists of two steps: encoding categorical variables using the custom encoder `CatToIntTransformer` and `LGBMRegressor`:

```
def train(
    self, X_train: pd.DataFrame, y_train: pd.Series, parameters: dict = None
) -> None:
    """Prepare features and train the model.

    :param X_train: Training features as a DataFrame
    :param y_train: Training target as a Series
    """

    class CatToIntTransformer(BaseEstimator, TransformerMixin):
        """
```

Transformer that encodes categorical columns as integer codes for LightGBM.
Unknown categories at transform time are encoded as -1.
"""

```
def __init__(self, cat_features: list[str]) -> None:
    self.cat_features = cat_features
    self.cat_maps_ = {}

def fit(self, X: pd.DataFrame, y=None) -> None:
    self.fit_transform(X)
    return self

def fit_transform(self, X: pd.DataFrame, y=None) -> pd.DataFrame:
    X = X.copy()
    for col in self.cat_features:
        c = pd.Categorical(X[col])
        # Build mapping: {category: code}
        self.cat_maps_[col] = dict(
            zip(c.categories, range(len(c.categories)), strict=False))
    X[col] = (
        X[col]
        .map(lambda val, col=col: self.cat_maps_[col].get(val, -1))
        .astype("category"))
    return X

def transform(self, X: pd.DataFrame) -> pd.DataFrame:
    X = X.copy()
    for col in self.cat_features:
        X[col] = (
            X[col]
            .map(lambda val, col=col: self.cat_maps_[col].get(val, -1))
            .astype("category"))
    return X

if parameters is not None:
    self.parameters = parameters
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", CatToIntTransformer(self.cat_features), self.cat_features)],
    remainder="passthrough")
self.pipeline = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("regressor", LGBMRegressor(**self.parameters))])
logger.info("Starting training...")
self.pipeline.fit(X_train, y_train)
```

LightGBM supports integer-encoded categorical features, which often perform better than one-hot encoding; consult the [LightGBM documentation](#)

for more details. We also need a custom encoder to make sure the LightGBM model treats integer-encoded features as categorical features and earlier unseen categories get assigned a value of `-1` to prevent errors while computing predictions.

Note that the `CatToIntTransformer` class is defined inside the `train` method of the `LightGBMModel` class. While this isn't ideal from a design standpoint, it helps avoid the need to make the `hotel_booking` package available when serving the model. This keeps the model self-contained and easier to deploy. We'll explore better ways to handle private dependencies when we discuss custom `pyfunc` models later.

Now that we've defined the model class and covered the relevant methods, let's take a look at how Ray Tune works. [Figure 3-5](#) shows the six main components of Ray Tune.¹ We begin by defining the search space, what objective to tune, the search algorithm, and when to stop. Then, we run the experiments and analyze the results.

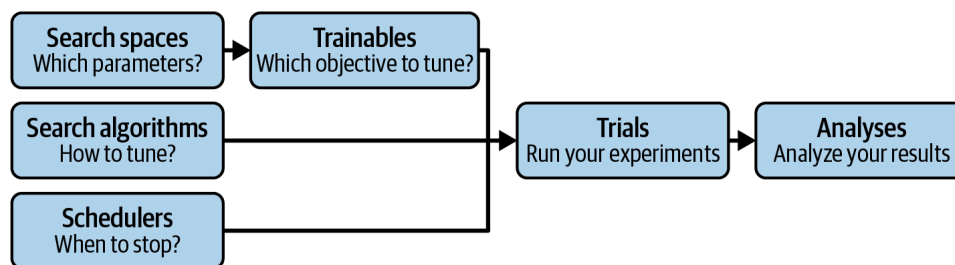


Figure 3-5. Key components of Ray Tune (adapted from Ray documentation)

Continuing with the `chapter3_2.model_finetuning.py` notebook, we define three hyperparameters to tune: `n_estimators`, `max_depth`, and `learning_rate`. Ray Tune supports a large variety of search space functions, and here, we use `tune.choice()` to sample `max_depth` uniformly from the specified choices, `tune.loguniform` to sample `learning_rate`, and `tune.qrandint` to sample `n_estimators`:

```
from ray import tune

param_space = {
    "n_estimators": tune.qrandint(50, 700, q=50),
    "learning_rate": tune.loguniform(0.01, 0.2),
    "max_depth": tune.choice([3, 5, 10, 15])}
```

The `tune.qrandint` function samples integers in fixed steps of size `q` within the specified range. For example, with interval `[50, 700]` and `q=50`, it draws the `n_estimators` value from `{50, 100, 150, ..., 650, 700}`.

This is convenient because it lets us avoid wasting trials on near-duplicates.

For `max_depth`, we also provide a small set of sensible choices that are sampled using `tune.choice`. A value of `-1` (unlimited) is also provided as an option. The sampling strategy for `max_depth` and `n_estimators` allows us to keep the number of unique configurations manageable, thus reducing the runtime.

Log-uniform distribution is useful for sampling the values that vary over several orders of magnitude, which makes `tune.loguniform` a perfect fit for the `learning_rate` hyperparameter.

Let's now define a *trainable*, where we pass the `train_with_nested_mlflow` function. Within that function, we start a nested run that takes `parent_run_id` and `experiment_id` as arguments, and then, we train the model and log the metrics. Config is passed by Ray Tune and contains the chosen hyperparameters:

```
import mlflow
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from hotel_booking.models.lightgbm_model import LightGBMModel

def train_with_nested_mlflow(config, X_train: pd.DataFrame,
                              X_valid: pd.DataFrame,
                              y_train: pd.DataFrame,
                              y_valid: pd.DataFrame,
                              project_config: ProjectConfig,
                              experiment_id: str,
                              parent_run_id: str):

    n_estimators, max_depth, learning_rate = (
        config["n_estimators"],
        config["max_depth"],
        config["learning_rate"],
    )
    with mlflow.start_run(experiment_id=experiment_id,
                          run_name=f"trial_n{n_estimators}_md{max_depth}_lr{learning_rate}",
                          nested=True, parent_run_id=parent_run_id
    ):
        model = LightGBMModel(config=project_config)
        model.train(
            X_train=X_train,
            y_train=y_train,
            parameters={
```

```

        "n_estimators": n_estimators,
        "max_depth": max_depth,
        "learning_rate": learning_rate
    }
)
y_pred = model.pipeline.predict(X_valid)
mse = mean_squared_error(y_valid, y_pred)
rmse = np.sqrt(mse)
metrics = {
    "mse": mse,
    "rmse": rmse,
    "mae": mean_absolute_error(y_valid, y_pred),
    "r2_score": r2_score(y_valid, y_pred),
}
mlflow.log_params(config)
mlflow.log_metrics(metrics)
tune.report(metrics)

```

In the `train_with_nested_mlflow` function, we must establish a connection with MLflow to log metrics and parameters. To accomplish that, we must set up the `DATABRICKS_HOST` and `DATABRICKS_TOKEN` environment variables. We also need to be aware of the lifetime of the token; it should not expire before the hyperparameter tuning experiment stops. We can retrieve the Databricks token and host by using the workspace client:

```

from databricks.sdk import WorkspaceClient

w = WorkspaceClient()
os.environ["DATABRICKS_HOST"] = w.config.host
os.environ["DATABRICKS_TOKEN"] = w.tokens.create(
    lifetime_seconds=1200).token_value

```

You can set up a Ray cluster on Databricks to utilize all the worker nodes (not just the driver node) in your Databricks cluster for hyperparameter optimization. Note that this operation does not make sense on a single node cluster (your cluster must have a sufficient number of workers), and it also does not work on a serverless compute or outside of a Databricks environment.

NOTE

At the time of writing this book, this code is not supported on serverless compute. Therefore, we also cannot execute it in Databricks Free Edition.

You'll need to use the `ray.util.spark.setup_ray_cluster()` command and define the `RAY_ADDRESS` environment variable so that all the processes can find and connect to the same Ray cluster:

```
from ray.util.spark import setup_ray_cluster

ray_conf = setup_ray_cluster(min_worker_nodes=2, max_worker_nodes=8)
os.environ['RAY_ADDRESS'] = ray_conf[0]
```

Now, we can set the experiment, start the MLflow parent run, and define and fit the tuner. Here, we use the `tune.with_parameters()` function, which allows us to pass constant, non-search space arguments to the trainable. It will store objects like datasets in the Ray object store and retrieve them when calling the function. Notice that we pass `experiment_id`, `parent_run_id`, project configuration, and datasets for training and validation using this function.

As part of the tuner, we define the `TuneConfig`, where the search algorithm is specified. In this example, we use Optuna with its default sampler, the TPE algorithm—a Bayesian optimization method that works well for complex and mixed hyperparameter search spaces. It may be less effective for very low-dimensional or purely continuous search spaces. Consult the [Optuna documentation](#) for more details. Here, we minimize the root mean square error (RMSE) as the chosen evaluation metric:

```
from ray.tune.search.optuna import OptunaSearch

mlflow.set_experiment("/Shared/hotel-booking-finetuning")
experiment_id = mlflow.get_experiment_by_name(
    "/Shared/hotel-booking-finetuning").experiment_id

n_trials = 5

with mlflow.start_run(
    run_name=f"optuna-finetuning-{datetime.now().strftime('%Y-%m-%d')}",
    tags={"git_sha": "1234567890abcd", "branch": "main"},
    description="LightGBM hyperparameter tuning with Ray & Optuna"
) as parent_run:

    tuner = tune.Tuner(
        tune.with_parameters(
            train_with_nested_mlflow,
            X_train = X_train,
            y_train = y_train,
            X_valid = X_valid,
            y_valid = y_valid,
```

```

        project_config=cfg,
        parent_run_id=parent_run.info.run_id,
        experiment_id=experiment_id,
    ),
    tune_config=tune.TuneConfig(
        search_alg=OptunaSearch(metric="rmse", mode="min"),
        num_samples=n_trials,
    ),
    param_space=param_space,
)
results = tuner.fit()

```

After we perform hyperparameter tuning, we can get the model parameters that belong to the best run. Those will be stored in the *project.config.yml* file:

```

best_result = results.get_best_result(
    metric="rmse", mode="min")
best_result.config

```

We can also use the UI to view all the nested runs under the parent run that belongs to the `hotel-booking-finetuning` experiment (see [Figure 3-6](#)).

Run Name	Created 	Dataset	Duration	Source
  optuna-finetuning-2026-02-23	 4 hours ago	-	11.4min	 chapter...
 trial_n200_md3_lr0.020912409950187706	 4 hours ago	-	13.0s	 default...
 trial_n600_md3_lr0.013024031587328476	 4 hours ago	-	21.1s	 default...
 trial_n650_md3_lr0.013108246155055186	 4 hours ago	-	11.7s	 default...
 trial_n650_md3_lr0.01337553107044523	 4 hours ago	-	16.1s	 default...
 trial_n50_md3_lr0.010084573575141574	 4 hours ago	-	3.3s	 default...
 trial_n150_md3_lr0.01651610252399948	 4 hours ago	-	3.3s	 default...
 trial_n150_md3_lr0.01652242152059382	 4 hours ago	-	13.3s	 default...
 trial_n300_md10_lr0.012003628334341319	 4 hours ago	-	27.0s	 default...
 trial_n300_md10_lr0.012253008819670083	 4 hours ago	-	9.7s	 default...
 trial_n300_md3_lr0.012003245040962342	 4 hours ago	-	7.8s	 default...
 trial_n150_md3_lr0.020736411746107446	 4 hours ago	-	4.1s	 default...
 trial_n150_md3_lr0.011788800210769617	 4 hours ago	-	4.9s	 default...
 trial_n150_md3_lr0.016686849813457954	 4 hours ago	-	6.2s	 default...
 trial_n50_md3_lr0.018127980467032265	 4 hours ago	-	3.1s	 default...
 trial_n50_md3_lr0.017722064621688793	 4 hours ago	-	1.7s	 default...
 trial_n100_md3_lr0.017836288238356073	 4 hours ago	-	3.5s	 default...
 trial_n100_md3_lr0.029996731680860016	 4 hours ago	-	6.1s	 default...
 trial_n100_md3_lr0.0269262441336487	 4 hours ago	-	6.3s	 default...
 trial_n150_md3_lr0.18477087060925015	 4 hours ago	-	3.7s	 default...
 trial_n150_md3_lr0.04702897002884801	 4 hours ago	-	2.8s	 default...

Figure 3-6. View of the nested run in the UI

It's also possible to plot and compare metrics that belong to different child runs (see [Figure 3-7](#)).

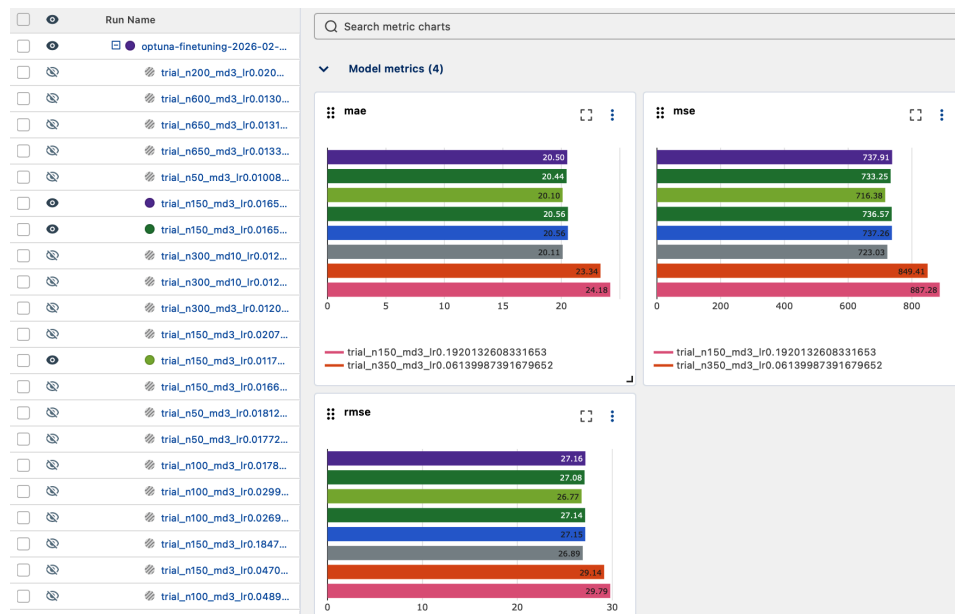


Figure 3-7. Comparing metrics across child runs

At this point, we have gone through experiment tracking, which is the backbone of everything that happens within MLflow. We've logged metrics, parameters, and various artifacts—but we have not yet logged a model, which is the next logical step after experiment tracking.

Logging and Registering Models with MLflow

Once you've tracked your experiments and identified a high-performing run, your next critical step is to save and package the trained model for future use. To let you go beyond simply saving a file, MLflow provides a standardized format called an MLflow Model. This convention defines how a model, its dependencies, and its code are stored. This is essential for downstream tasks like real-time serving, which is a topic we'll cover in [Chapter 4](#).

MLflow supports a wide range of flavors “out-of-the-box.” Here are some popular ones:²

- `mlflow.lightgbm`
- `mlflow.prophet`
- `mlflow.pytorch`
- `mlflow.sklearn`
- `mlflow.spacy`

- `mlflow.tensorflow`
- `mlflow.xgboost`

You can log a model by using the `mlflow.<model_flavor>.log_model()` function.

To demonstrate logging, we'll use a LightGBM model, even though we've covered the specific model code previously. We will log this model with the `sklearn` flavor rather than `lightgbm` because it's integrated within a scikit-learn pipeline.

Model Logging with the sklearn Flavor

Much like how we defined the training and validation split, we'll now define the train and test sets. The train set now contains 12 months of data, ending on 2018-11-30, while the test set will contain data exclusively from December 2018. Additionally, we'll use the latest version of the `hotel_booking` table:

```
import json
from datetime import datetime

import mlflow
from mlflow import MlflowClient
from mlflow.models import infer_signature
from pyspark.sql import SparkSession

from hotel_booking.config import ProjectConfig, Tags
from hotel_booking.data.data_loader import DataLoader
from hotel_booking.models.lightgbm_model import LightGBMModel
from hotel_booking.utils.common import set_mlflow_tracking_uri

set_mlflow_tracking_uri()
cfg = ProjectConfig.from_yaml(config_path="../project_config.yml")

spark = SparkSession.builder.getOrCreate()
data_loader = DataLoader(spark=spark, config=cfg)
X_train, y_train, X_test, y_test = data_loader.split()
```

Later on in the script, we use these:

- `train_query` and `test_query`, which contain a query that's used to retrieve the data for training and testing (attributes of the data loader object)
- The `train_set_spark` and `test_set_spark` PySpark DataFrames (attributes of the data loader object)

- `X_train`, `y_train`, `X_test`, and `y_test`

We begin by training the model, using the same method we used earlier during hyperparameter tuning:

```
model = LightGBMModel(config=cfg)
model.train(X_train=X_train, y_train=y_train)
```

Then, we set the MLflow experiment, start an MLflow run, and log parameters:

```
mlflow.set_experiment("/Shared/hotel-booking-training")
tags = Tags(**{"git_sha": "1234567890abcd", "branch": "main"})

run = mlflow.start_run(
    run_name=f"lightgbm-training-{datetime.now().strftime('%Y-%m-%d')}",
    description="LightGBM model training",
    tags=tags.to_dict(),
)
run_id = run.info.run_id
mlflow.log_params(cfg.parameters)
```

Next, we want to proceed with logging the model, and to be able to register it later in Unity Catalog, we need to provide a model signature that defines the structure of the model input and output. The easiest way to do this is to use the `infer_signature` function from the `mlflow.models` module:³

```
signature = infer_signature(
    model_input=X_test, model_output=model.pipeline.predict(X_test)
)
```

To ensure full traceability and reproducibility, we must log the model input. To do this, we pass a PySpark dataframe and a specific SQL query that also contains the delta table version:

```
training = mlflow.data.from_spark(
    df=data_loader.train_set_spark,
    sql=data_loader.train_query
)
testing = mlflow.data.from_spark(
    df=data_loader.test_set_spark,
    sql=data_loader.test_query
)
```

```
mlflow.log_input(training, context="training")
mlflow.log_input(testing, context="testing")
```

Now, we can finally log the model. Here, we also provide an input example, which we can also use as an example for Model Serving inference later:

```
model_info = mlflow.sklearn.log_model(
    sk_model=model.pipeline,
    name="lightgbm-pipeline",
    signature=signature,
    input_example=X_test[0:1]
)
```

Notice that we have not computed and logged any model metrics yet. This time, we'll use MLflow Evaluation to compute the metrics. The `mlflow.models.evaluate()` function takes the model URI, evaluation data, target, model type ("regressor" and "classifier" are supported values), and evaluators as inputs. Here, we'll use the default evaluator, but you can define custom evaluators as well.

The `mlflow.models.evaluate()` function uses an active run to log metrics (or starts a new run if no active run is available):

```
eval_data = X_test.copy()
eval_data[project_config.target] = y_test

result = mlflow.models.evaluate(
    model_info.model_uri,
    eval_data,
    targets=project_config.target,
    model_type="regressor",
    evaluators=["default"],
)
mlflow.end_run()
```

For a regression task, the following default metrics are logged:

- score
- example_count
- mean_absolute_error
- mean_squared_error
- root_mean_squared_error
- sum_on_target

- `mean_on_target`
- `r2_score, max_error`
- `mean_absolute_percentage_error`

For classification tasks, a different set of default metrics is logged, such as these:

- `precision_score`
- `recall_score`
- `f1_score`
- `roc_auc`

In this book, we focus on demonstrating a regression task and will not cover classification examples. However, the underlying processes remain the same: training runs are tracked in MLflow, models and datasets can be logged, and evaluation metrics are automatically captured to help compare and select the best-performing models.

Viewing the Logged Model

We can now view our logged model that belongs to the `hotel-booking-training` experiment in the Models tab in the UI . We can view all the computed metrics and logged parameters in the Overview tab (see [Figure 3-8](#)).

Description

No description

Metrics (10)

Q Search metrics			
Metric	Dataset	Source run	Value
score	 dataset (#23978ef5)	lightgbm-training-...	0.0862226783236...
example_count	 dataset (#23978ef5)	lightgbm-training-...	1170
mean_absolute_error	 dataset (#23978ef5)	lightgbm-training-...	26.05431643753705
mean_squared_error	 dataset (#23978ef5)	lightgbm-training-...	1072.940183078334
root_mean_squared_error	 dataset (#23978ef5)	lightgbm-training-...	32.7557656463459

Parameters (3)

Q Search parameters	
Parameter	Value
learning_rate	0.05
max_depth	3
n_estimators	200

Figure 3-8. Logged model view in the UI

We can find logged artifacts in the Artifacts tab. Under “lightgbm-pipeline,” we can find the *MLmodel* file that defines the flavors we can use the model with. There is always a `python_function` flavor that describes how to run the model as a Python function, even if the model is logged with a `sklearn` flavor. The `python_function` flavor is used for Model Serving. In our case, here’s how we log the flavors in the *MLmodel* file:

```
flavors:
  python_function:
    code: code
    env:
      conda: conda.yaml
      virtualenv: python_env.yaml
    loader_module: mlflow.sklearn
    model_path: model.pkl
    predict_fn: predict
    python_version: 3.12.12
  sklearn:
    code: code
    pickled_model: model.pkl
    serialization_format: cloudpickle
    sklearn_version: 1.7.0
```

Fields in the *MLmodel* file include `artifact_path`, runtime information (if available), the MLflow version, `model_id`, the MLflow `run_id`, the model signature in JSON format, and the timestamp of when the model was logged. All this information is also available in the `ModelInfo` object that gets returned when we run the `mlflow.log_model` command.

Next to the *MLmodel* files, we'll find the following logged artifacts:

model.pkl

This is the serialized model object.

conda.yaml

This file defines the conda environment: when logging a model, we can pass a conda environment parameter with (additional) model dependencies.

python_env.yaml

This file contains information that we need to restore a model environment using virtualenv: Python version, build dependencies, and reference to the project requirements.

requirements.txt

This file contains inferred project requirements. It's the pip portion of the *conda.yml* file.

input_example.json

This file contains a model input example.

serving_input_example.json

This file contains a sample payload that we can use to send requests once we've deployed the model behind an endpoint.

LoggedModel

This object was introduced with MLflow 3. It allows users to interact with the logged model outside of the experiment run. We've added these functions:

```
mlflow.get_logged_model()
```

This allows us to retrieve a logged model by its `model_id`.

```
mlflow.last_logged_model()
```

This allows us to retrieve the most recently logged model.

```
mlflow.search_logged_models()
```

This allows us to search for logged models, in much the same way we can search for experiment runs.

Let's retrieve the model using the `mlflow.get_logged_model()` function:

```
logged_model = mlflow.get_logged_model(model_info.model_id)
```

We can now inspect the `LoggedModel` object by saving all class attributes in a JSON file:

```
import json
logged_model_dict = logged_model.to_dictionary()
logged_model_dict["metrics"] = [x.__dict__ for x in logged_model_dict["metrics"]]
with open("../demo_artifacts/logged_model.json", "w") as json_file:
    json.dump(logged_model_dict, json_file, indent=4)
```

And then, we can visualize it with JSON Lens or a similar VS Code extension (see [Figure 3-9](#)).

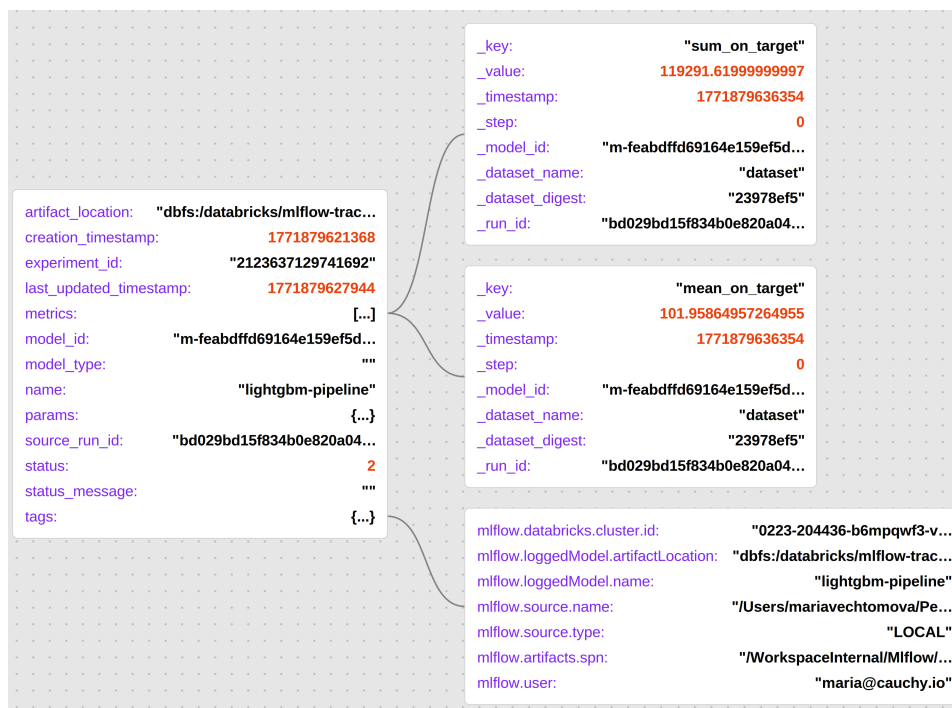


Figure 3-9. Visualizing the LoggedModel class

Notice that model metrics and model parameters are now attributes of the `LoggedModel` class, so we can access it in the following way:

```
logged_model.params
logged_model.metrics
```

Previously, it was only possible to retrieve metrics and parameters through experiment runs. Similarly, we can now also load a model by passing a `model_id` in the model URI (which we could previously only do by using the `run_id`):

```
model = mlflow.sklearn.load_model(f"models://{model_info.model_id}")
```

While logging the model, we've also logged the input train and test datasets. We can load the logged datasets at any point later, as long as the table version used for logging is within the retention period:

```
run = mlflow.get_run(run_id)

inputs = run.inputs.dataset_inputs
training_input = next(
    (x for x in inputs if len(x.tags) > 0 and x.tags[0].value == 'training'),
    None)
training_source = mlflow.data.get_source(training_input)
training_source.load()

testing_input = next(
    (x for x in inputs if len(x.tags) > 0 and x.tags[0].value == 'testing'),
    None)
testing_source = mlflow.data.get_source(testing_input)
testing_source.load()
```

Registering the Model

To use the logged model for Model Serving on Databricks or make it accessible to other teams, we must register the model in Databricks Unity Catalog.

We can do this by providing the `registered_model` argument to the `log_model()` function. In that case, the `ModelInfo` object will also have a `registered_model_version` attribute. However, we may not want to register the model right away without first evaluating it against the latest registered version of the model. Since no model is registered yet, let's now register a model using the `mlflow.register_model()` function:

```

model_name = (
    f"{cfg.catalog}.{cfg.schema}.hotel_booking_basic"
)
registered_model = mlflow.register_model(
    model_uri=logged_model.model_uri,
    name=model_name,
    tags=tags.to_dict(),
)

```

We'll demonstrate the evaluation process in the next section of this chapter.

We can also register the model using the MLflow client. In this case, we can pass the `deployment_job_id` as an argument for the `create_registered_model` function, which can facilitate a model approval flow using the [MLflow 3 deployment jobs functionality](#):

```

from mlflow.tracking.client import MlflowClient

client = MlflowClient()
job_id = "1234567890abcdef" # Example job ID
registered_model = client.create_registered_model(model_name, deployment_job_id=

```

Passing a non-existing `job_id` will result in an error, as the MLflow client checks whether the job exists in the background.

After we register the model, we can assign an alias that refers to a particular version of a registered model. We can use an alias in a model URI to simplify the way we can access the registered model.

We also find it convenient to assign the `"latest-model"` tag to the most recently registered model. We can't use the `"latest"` alias directly because it's reserved, nor can we use `"latest"` in the model URI:

```

client.set_registered_model_alias(
    name=model_name,
    alias="latest-model",
    version=registered_model.version,
)

model = mlflow.sklearn.load_model(f"models:{model_name}@latest-model")

```

It's a good idea to use aliases, as model registry search functionality is not widely supported on Unity Catalog. For instance, it's only possible to search for model versions by name:

```
v = mlflow.search_model_versions(  
    filter_string=f"name='{model_name}'")
```

We also can't search for model versions using filter strings. For instance, this would not work:

```
# not supported  
v = mlflow.search_model_versions(  
    filter_string="tags.git_sha='1234567890abcd'")
```

However, we can get the model version by using an alias:

```
model_version = client.get_model_version_by_alias(  
    alias="latest-model", name=model_name)
```

Wrapping the Model Using pyfunc

The model signature in MLflow defines the way different interfaces interact with the model. For instance, it defines the payload of the endpoint if the model gets served with Databricks Model Serving. (We'll go into detail on that in [Chapter 4](#).)

We previously registered a `sklearn` pipeline. If we deploy it behind an endpoint now, we will get an output in the format: `{"Predictions": [120.1234567]}`. We may want to have a custom output, if so, and a `pyfunc` model flavor will come in handy.

There are other situations in which `pyfunc` becomes important:

- When we want to use an ensemble of models trained separately and serve it behind an endpoint
- When Model Serving requires a specific context (for instance, a file that defines what predictions must be adjusted)
- When we need to access other systems (for example, a vector database) to return predictions

Essentially, we are using `pyfunc` as a wrapper. Keeping the definition of the payload separate from the model itself is quite useful: we can easily adjust the `pyfunc` wrapper definition without touching the registered model itself. In a certain sense, it's very similar to the functionality of a FastAPI wrapper.

We defined the `HotelBookingModelWrapper` class under the `models` module of the `hotel_booking` package. It loads a `sklearn` pipeline as part of the `load_context()` method and modifies the predictions. Here, we are adding a 5% commission on top of the price and rounding the total price down to two decimals:

```
from datetime import datetime

import mlflow
import numpy as np
import pandas as pd
from mlflow import MlflowClient
from mlflow.models import infer_signature
from mlflow.models.model import ModelInfo
from mlflow.pyfunc import PythonModelContext
from mlflow.utils.environment import _mlflow_conda_env

from hotel_booking.config import Tags


def adjust_price(price: float) -> float:
    """Adjust the price by a fixed percentage."""
    return round(price * 1.05, 2)


class HotelBookingModelWrapper(mlflow.pyfunc.PythonModel):

    def load_context(self, context: PythonModelContext) -> None:
        self.model = mlflow.sklearn.load_model(
            context.artifacts["lightgbm-pipeline"]
        )

    def predict(self, context: PythonModelContext,
                model_input: pd.DataFrame | np.ndarray) -> dict:
        preds = self.model.predict(model_input)
        return {"Total price per night": [adjust_price(pred) for pred in preds]}
```

Notice that the `adjust_price` function is defined outside of the class, and it will not be accessible to the model at the serving step unless we log our custom package using `code_paths` and `conda_env`. This is the cleanest way to log the package and make sure all the modules are accessible, no matter how your code is structured.

The `HotelBookingModelWrapper` class has the `log_register_model()` method. It requires the `ModelInfo` object from the model we are going to wrap, the model name we'll use to register the `pyfunc` model, the experiment name, tags, and `code_paths`:

```

def log_register_model(self, wrapped_model_info: ModelInfo,
                        pyfunc_model_name: str,
                        experiment_name: str, tags: Tags,
                        code_paths: list) -> None:
    """Log and register the Pyfunc model."""
    wrapped_model_uri = wrapped_model_info.model_uri
    input_example_dict = mlflow.artifacts.load_dict(
        f"{wrapped_model_info.artifact_path}/input_example.json")
    input_example = pd.DataFrame(input_example_dict['data'],
                                columns=input_example_dict['columns'])

    additional_pip_deps = []
    for package in code_paths:
        whl_name = package.split("/")[-1]
        additional_pip_deps.append(f"code/{whl_name}")
    conda_env = _mlflow_conda_env(additional_pip_deps=additional_pip_deps)

    mlflow.set_experiment(experiment_name=experiment_name)
    ts = datetime.now().strftime('%Y-%m-%d')
    with mlflow.start_run(run_name=f"wrapper-lightgbm-{ts}",
                          tags=tags.to_dict()):

        signature = infer_signature(
            model_input=input_example,
            model_output={'Total price per night': [100.00]})
        self.model_info = mlflow.pyfunc.log_model(
            python_model=self,
            name="pyfunc-wrapper",
            artifacts={
                "lightgbm-pipeline": wrapped_model_uri},
            signature=signature,
            code_paths=code_paths,
            conda_env=conda_env,
            input_example=input_example
        )

    client = MlflowClient()
    registered_model = mlflow.register_model(
        model_uri=self.model_info.model_uri,
        name=pyfunc_model_name,
        tags=tags.to_dict())
    client.set_registered_model_alias(
        name=pyfunc_model_name,
        alias="latest-model",
        version=registered_model.version,
    )
    return registered_model.version

```

There are a couple of points to pay attention to. First of all, we infer the signature using the input example from the basic model. When logging

the model, we need to pass `code_paths` and `conda_env` arguments. The `code_paths` list contains a local path to the `hotel_booking` package wheel, and `conda_env` contains the location of the wheel in the artifacts folder (it will be saved in the code folder).

To provide the `code_paths`, we had to build a wheel using the `uv build` command first. We can pass the wheel's location as an argument if we run this code as part of a pipeline we deployed through a DAB. The bundle automatically builds and uploads the wheel to a location within its root folder:

```
__version__ = version("hotel_booking")
code_paths = [f"../dist/hotel_booking-{__version__}-py3-none-any.whl"]
```

Finally, we introduced Pydantic validation for tags. Now, let's initialize, log, and register the wrapper:

```
from hotel_booking.models.pyfunc_model_wrapper import HotelBookingModelWrapper

wrapper = HotelBookingModelWrapper()
pyfunc_model_name = f"{cfg.catalog}.{cfg.schema}.hotel_booking_pyfunc"

wrapper.log_register_model( #Defined earlier when logging model with sklearn fla
    wrapped_model_info=model_info,
    pyfunc_model_name=pyfunc_model_name,
    experiment_name="/Shared/hotel-booking-wrapper",
    tags=tags,
    code_paths=code_paths,
)
```

A `pyfunc` wrapper works in the following way:

- When logging the model, we provide an artifacts argument, which is a dictionary containing the paths of artifacts needed for the model.
- The artifacts dictionary is used in the `load_context()` method, which runs at the `mlflow.pyfunc.log_model()` step. The `load_context()` method downloads the referenced artifacts and stores them together with the wrapper (see [Figure 3-10](#)).

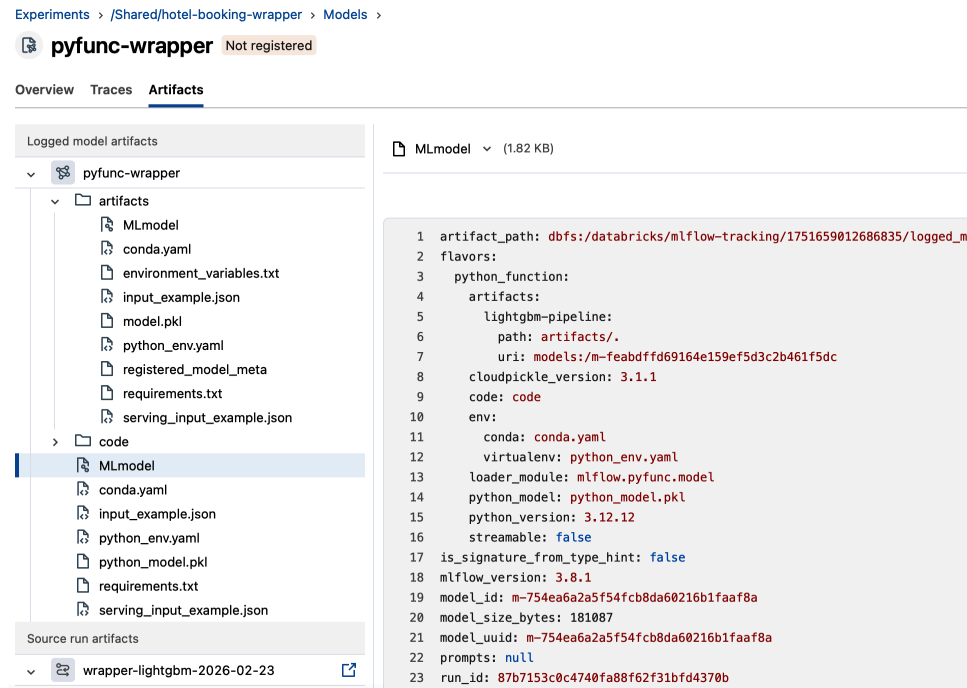


Figure 3-10. UI view of a logged `Pyfunc` wrapper model

The top-level *MLmodel* file has one flavor (`pyfunc`), and it includes a reference to the wrapped model artifact location:

```

flavors:
  python_function:
    artifacts:
      lightgbm-pipeline:
        path: artifacts/.
        uri: models:/mlops_dev.hotel_booking.hotel_booking_basic@latest-model

```

If we load the model, we will get back an object of the `PythonModel` type:

```
loaded_pyfunc_model = mlflow.pyfunc.load_model(pyfunc_model.model_uri)
```

When we load the model, we can use the `predict` function without passing the context:

```
loaded_pyfunc_model.predict(X_test[0:1])
```

However, this is not the `HotelBookingModelWrapper` object we logged. To get our object back and be able to access underlying attributes, we need to unwrap the loaded model:

```
unwrapped_model = loaded_pyfunc_model.unwrap_python_model()
```

To run the `predict` function, we need to pass context here (just with a value of `None`):

```
unwrapped_model.predict(context=None, model_input=X_test[0:1])
```

Earlier in the chapter, we mentioned that we do not want to register models right away every time we log them. First, we need to make sure that the model has actually improved since the previous iteration, which leads us to the evaluation step.

MLflow Evaluation

To evaluate the impact of updated data and compare a newly trained model with the previous one, we need additional data. Since we are working with a static dataset, we'll simulate the arrival of new data by generating a synthetic dataset that represents a full month of hotel bookings.

The `DataProcessor` class includes a `generate_synthetic_df()` method, which takes the number of observations to generate and an optional `max_date` parameter that indicates the timestamp after which the new data should be created. If `max_date` is not provided, it is retrieved from the `hotel_booking` table.

We use the `sdv` package and the `GaussianCopulaSynthesizer` to generate data with a distribution similar to that of the original dataset, and then, we adjust the reservation and arrival dates:

```
import calendar
import datetime
import random
from datetime import datetime, timedelta

import pandas as pd
from dateutil.relativedelta import relativedelta
from pyspark.sql import SparkSession
from sdv.metadata import Metadata
from sdv.single_table import GaussianCopulaSynthesizer

from hotel_booking.config import ProjectConfig


class DataProcessor:
    """A class for preprocessing and managing DataFrame operations.
```

```
This class handles data preprocessing and saving to UC tables.
"""
```

```
def __init__(self: "DataProcessor",
             df: pd.DataFrame,
             config: ProjectConfig,
             spark: SparkSession) -> None:
    self.df = df
    self.cfg = config
    self.spark = spark
    self.target_table = f"{self.cfg.catalog}.{self.cfg.schema}.hotel_booking"

def preprocess(self: "DataProcessor") -> None:
    """Preprocess the DataFrame."""
    self.df.columns = self.df.columns.str.replace(r"[ -]", "_", regex=True)
    # Code to clean the data
    self.df["arrival_date"] = self.df["date_of_reservation"] + pd.to_timedelta(
        self.df["lead_time"], unit="d")
    self.df["arrival_month"] = self.df["arrival_date"].dt.month

def generate_synthetic_df(self: "DataProcessor",
                         cn: int = 1000, max_date: datetime = None) -> None:
    """Preprocess the DataFrame stored in self.df."""
    # Code to generate the data

def save_to_catalog(self: "DataProcessor") -> None:
    """Preprocess the DataFrame stored in self.df."""
    self.spark.createDataFrame(self.df).write.mode("append").saveAsTable(
        f"{self.target_table}")
    self.spark.sql(
        f"""ALTER TABLE {self.target_table} SET TBLPROPERTIES
            (delta.enableChangeDataFeed = true);"""
    )
```

For implementation details, refer to the

`hotel_booking.data.data_processor` module.

Now, take a look at the notebook called *chapter3_4.model_evaluation.py*.

We start by generating synthetic data and storing it via Unity Catalog:

```
import pandas as pd
from pyspark.sql import SparkSession

from hotel_booking.config import ProjectConfig, Tags
from hotel_booking.data import DataLoader, DataProcessor

from hotel_booking.utils.common import set_mlflow_tracking_uri
```

```

set_mlflow_tracking_uri()
spark = SparkSession.builder.getOrCreate()

cfg = ProjectConfig.from_yaml("../project_config.yml")

# Load and process the data
df = pd.read_csv("../data/booking.csv")
data_processor = DataProcessor(df=df, config=cfg, spark=spark)
data_processor.preprocess()
data_processor.generate_synthetic_df(n=1000, max_date=None)
data_processor.save_to_catalog()

```

Then, we load the data (where the test set includes the newly generated month of data and the train set 12 months before) and train the model:

```

from hotel_booking.models.lightgbm_model import LightGBMModel

data_loader = DataLoader(spark=spark, config=cfg)
X_train, y_train, X_test, y_test = data_loader.split()

model = LightGBMModel(config=cfg)

model.train(X_train=X_train,
            y_train=y_train)

```

The code for model logging will then be added to the `LightGBMModel` class. This is how we can use it:

```

tags=Tags(**{"git_sha": "1234567890abcd", "branch": "main"})

model_info = model.log_model(
    experiment_name="/Shared/hotel-booking-training",
    tags=tags,
    X_test=X_test,
    y_test=y_test,
    train_set_spark=data_loader.train_set_spark,
    train_query=data_loader.train_query,
    test_set_spark=data_loader.test_set_spark,
    test_query=data_loader.test_query
)

```

We can access metrics of the newly logged model through the model object:

```

metrics_new = model.metrics

```

Let's now evaluate the latest currently registered model against the new test set:

```
import mlflow

sklearn_model_name = f"{cfg.catalog}.{cfg.schema}.hotel_booking_basic"
model_uri = f"models://{sklearn_model_name}@latest-model"
eval_data = X_test.copy()
eval_data[cfg.target] = y_test

result = mlflow.models.evaluate(
    model_uri,
    eval_data,
    targets=cfg.target,
    model_type="regressor",
    evaluators=["default"],
)
metrics_old = result.metrics
```

Previously, we were evaluating model performance based on the RMSE. If this metric for the new model is lower than for the old model, we can proceed with registering the model:

```
if metrics_new['root_mean_squared_error'] < metrics_old['root_mean_squared_error']:
    model.register_model(model_name=sklearn_model_name, tags=tags)
```

The logic for registering the model is then added to the `LightGBMModel` class:

```
def register_model(self: "LightGBMModel", model_name: str, tags: Tags) -> None:
    """Register the model in MLflow Model Registry."""
    client = MlflowClient()
    registered_model = mlflow.register_model(
        model_uri=self.model_info.model_uri,
        name=model_name,
        tags=tags.to_dict(),
    )
    client.set_registered_model_alias(
        name=model_name,
        alias="latest-model",
        version=registered_model.version,
    )
    return registered_model.version
```

Now that we've explored MLflow experiment tracking, model logging, and model registration in depth, you need to be aware of the resource and API limits that apply, which may influence the way you organize your workspaces and experiments.

Resource and API Limits

As you scale your ML workflows on Databricks, you may encounter platform limits. For example, when you're running extensive hyperparameter tuning across multiple projects within the same workspace, each experiment can generate hundreds or thousands of runs that log metrics, parameters, artifacts, and models to MLflow. Over time, this can lead to throttling or failures if you exceed API rate limits or resource quotas.

For instance, distributed hyperparameter tuning with Ray Tune across several teams in a shared workspace can quickly approach the 500,000 runs per MLflow experiment limit.

In retail demand forecasting scenarios, in which you train a separate model in each product category, you'll commonly log hundreds or even thousands of models within a single experiment. In such cases, you may reach the 500,000 logged models per MLflow experiment limit if you do not clean it in a timely manner.

Understanding these constraints helps you plan more robust, scalable systems. [Table 3-1](#) outlines some of the key resource limits you should be aware of when using MLflow for model development and Unity Catalog for model registration.

Table 3-1. Resource limits for experiment tracking and registered models

Resource	Limit
Experiment tracking metric steps	10 M per MLflow run
Experiment tracking parameters	1,600 per MLflow run/ logged model
Experiment tracking tags	1,600 per MLflow run/ logged model; 1,000 per experiment
Experiment tracking runs	500 k per MLflow experiment
Experiment tracking logged models	500 k per MLflow experiment
Experiment tracking metrics	1,000 per MLflow run/ logged model
Registered model (Unity Catalog)	1,000 per schema; 5000 per metastore
Registered model alias (Unity Catalog)	10 per registered model
Model versions (Unity Catalog)	10 k per registered model; 100k per metastore

[Table 3-2](#) outlines some of the key API limits you should be aware of when using MLflow.

Table 3-2. MLflow API limits

API	Limit
<i>/mlflow/experiments/*</i> (except list and search)	120 rps per workspace
<i>mlflow/runs/</i> (except search)	
<i>/mlflow/metrics/get-history</i>	
<i>/mlflow/artifacts/list</i>	
<i>/mlflow/logged-models/</i> (except search and artifacts)	
<i>/mlflow/experiments/list</i>	7 rps per workspace
<i>/mlflow/experiments/search</i>	
<i>/mlflow/runs/search</i>	
<i>/mlflow/logged-models/search</i>	
<i>/mlflow/logged-models/{model_id}/artifacts/directories</i>	2 rps per workspace

For the full, up-to-date overview of the resource and API limits, consult the [Databricks documentation](#).

Conclusion

In this chapter, we went through the whole process of experimentation, choosing the right way to split the data for training and validation, hyperparameter tuning, logging, evaluating, and registering models.

By the end of the chapter, we created three experiments:

1. hotel-booking-finetuning
2. hotel-booking-training
3. hotel-booking-wrapper

Under each experiment, we created experiment runs with descriptive names, to make runs easy to find:

1. optuna-finetuning-yyyy-mm-dd
2. lightgbm-training-yyyy-mm-dd
3. wrapper-lightgbm-yyyy-mm-dd

We tagged all the experiment runs and registered models with `git_sha` (for now, a dummy value) and `branch`, so we can find the exact code version that's responsible for creating the artifacts. We logged the reference to the exact version of training and testing data used for model training and evaluation.

All this allows us to achieve full traceability and reproducibility for our deployments. [Chapter 5](#) will show you how to pass actual `git_sha`, `job_id`, and `run_id` as parameters to the scripts running in a production setting.

But before we can go into deployment, we need to talk about Model Serving architectures and implement a Model Serving endpoint for the use case we just went through.

¹ For more details, check out the [Ray documentation](#).

² You can find the full list in [the `mlflow.models` documentation](#).

³ To explore more detailed information about model signatures in MLflow as well as additional methods to define them, refer to [the documentation](#).

Previous chapter

< [2. Developing on
Databricks](#)

Next chapter

[4. Model Serving: Architectures and
Implementation](#)

>